School of Future Tech

Case Study

on

Recent Searches Tracker (State + Persistence Logic)

by

Dharmit Sathvara

150096725080

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to Case study

Recent Searches Tracker is a web-based application that records and manages the most recent search queries entered by a user. Many modern websites such as search engines, e-commerce platforms, and social media applications maintain a search history feature to improve user need.

This case study focuses on developing a **Recent Searches Tracker using HTML, CSS, and JavaScript**. The system stores the latest search queries entered by the user and displays them in a list format. The application maintains only the **last 10 unique searches** to keep the history organized and prevent unnecessary storage.

The system also uses **browser localStorage** to store search data so that the search history remains available even after the page is refreshed.

The objective of this case study is to understand concepts such as **JavaScript state management, array manipulation, DOM interaction, and browser storage**, which are essential for building modern web applications.

2. Problem Statement/Case Background

Background

Search history is an important feature used in many web applications. It helps users quickly revisit their previous searches without typing them again. However, managing search history efficiently requires proper logic to ensure that the stored searches remain unique and limited in number.

This case study focuses on implementing a **Recent Searches Tracker** that maintains a list of recent searches while ensuring uniqueness and limiting the number of stored entries.

Abstract

Recent Searches Tracker is a **JavaScript-based browser application** that stores the last **10 unique search queries** entered by a user. The system removes duplicate searches and moves repeated searches to the top of the list.

The application uses **arrays to manage search history**, **DOM manipulation to update the user interface**, and **localStorage for persistence** so that the data remains saved even after refreshing the page.

It also includes a **Clear History feature** that removes all stored search queries from both the application and the browser storage.

3. Problem Statement/Case Study Design

The System is designed a simple client-side web page that runs entirely in browser

Main components of System are:

Search input: A text input field where user can type their search queries

Search Button: A button used to trigger the search event and store the search query in the history.

Recent Search List: A list that displays the most recent search queries entered by the user.

Clear History Button: A button that allows users to delete all stored search history.

Workflow of the System

1. User enters a search query.
2. The system captures the input using JavaScript.
3. The input is normalized by removing extra spaces.
4. The search query is checked for duplicates.
5. The query is added to the top of the history array.
6. scroller moves to searched place and the card is highlighted at the borders.
7. If the history exceeds 10 entries, the oldest entry is removed.
8. The updated history is stored in localStorage.

9. The search history list is updated dynamically on the webpage.

4. Methods & Algorithm Technologies applied in the problem statement/Case study

It uses several web development technologies and program techniques

1. HTML

HTML is used to create the structure of the webpage including:

- Search input field
- Search button
- Recent search list
- Clear history button
- image cards

HTML provides the basic layout required for user interaction.

2. CSS

CSS is used to design and style the user interface.

It improves the visual appearance of the application by styling:

- Search bar
- Buttons
- History list
- Page layout
- Keyframe infinite Animation

CSS helps make the application more user-friendly and visually appealing.

3. JavaScript

JavaScript is used to implement the main functionality of the system such as:

- Capturing search input
- Managing search history
- Enforcing uniqueness
- Limiting history size
- Saving and retrieving data from localStorage
- Updating the user interface dynamically

User Search History Management Flow

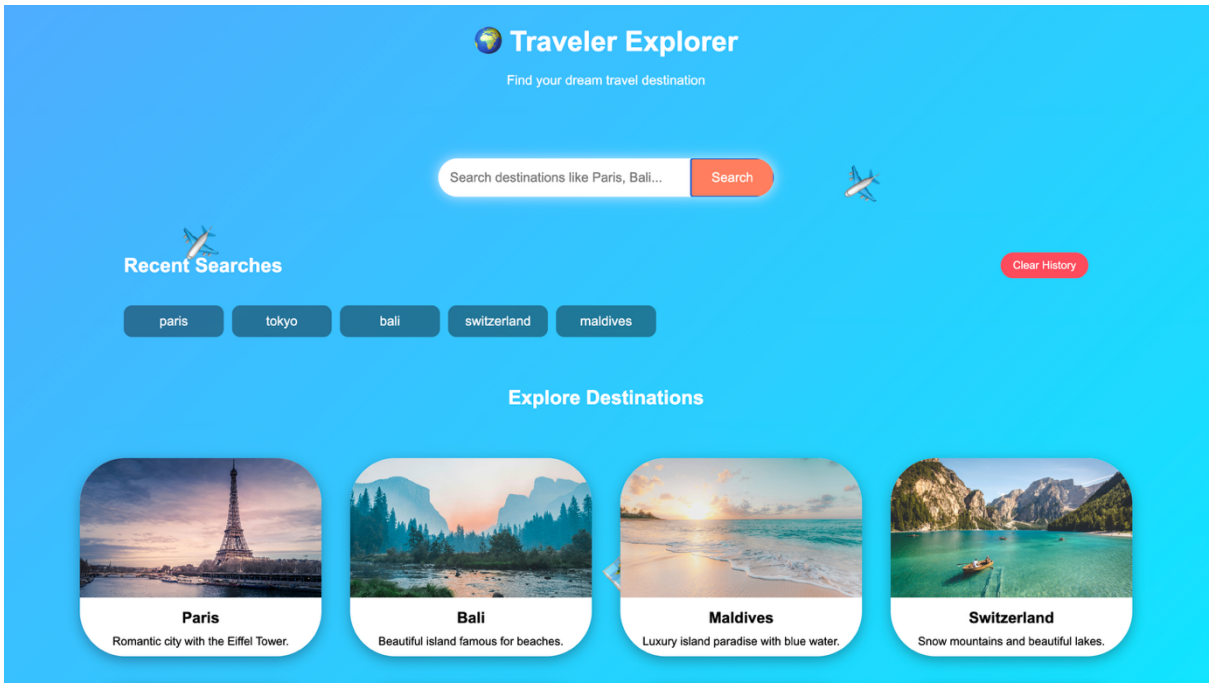
5. Problem Statement / Case Study Implementation Details and Snapshots

Implementation Steps

The application was implemented using the following steps:

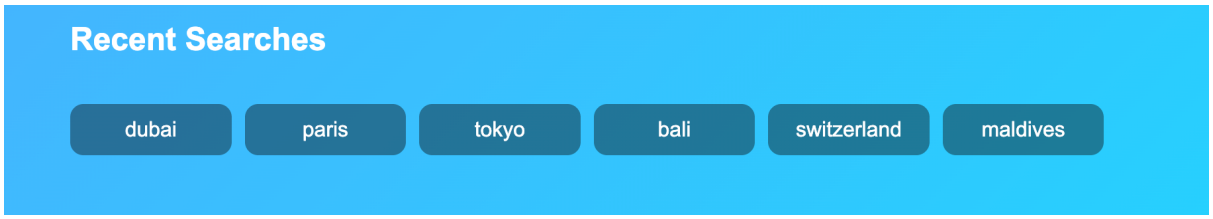
1. Create the HTML layout for search input , buttons, Image cards.
2. Design the user interface using CSS.
3. Implement JavaScript logic to capture search input.
4. Create an array to store search queries.
5. Apply uniqueness filtering using array methods.
6. Limit the search history to 10 entries.
7. Store search history using localStorage.
8. Restore history when the page reloads.
9. Implement the clear history feature.

OUTPUT



This is the full webpage on travel explorer containing Search input, search button, Recent Searches, Clear History button more cards below

This is the search input area with Search button ,after the searched over the search place will be added to recent searches



So as we searched recently Dubai , the recent search is shown Dubai and this search will last up to 10 entries after that automatically older search will be removed and new search will be added if user searches more than 10 places

This is the Clear History button after clicking on this the whole recent search history get disappear.

As mentioned before after clicking on clear history the whole history gets disappear and as there is no search now message shows no searches yet.

6. Problem Statement/ Case Study Results and Conclusion

The **Recent Searches Tracker** was successfully developed using **HTML, CSS, and JavaScript**. The system allows users to enter search queries and maintains a list of the **last 10 unique searches**.

The application stores search history using **localStorage**, which ensures that the data remains available even after refreshing the webpage. The system also includes a **Clear History feature** that removes all stored searches instantly.

Through this case study, important concepts such as **JavaScript array manipulation, DOM interaction, state management, and browser persistence** were successfully implemented.

Overall, the project demonstrates how JavaScript can be used to build **interactive and user-friendly web applications** with efficient search history management.

7. Reference:

- HTML,CSS,JS Documentation
- W3Schools.com(CSS)
- YouTube for animation(https://youtu.be/CZTCciHE72I?si=tfwJFpcFQZ53cdAq)
- YouTube (JavaScript)

Git Link: